

tests/memory/ proxy_memory_soak_test.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:**

Authored for the §11.4.169 memory/soak test-type coverage of the live HTTP forward proxy. Parse-clean under `sh -n` AND `bash -n` (§11.4.67). Honest `topology_unsupported` SKIP when the proxy-squid container (or cgroup memory accounting) is absent — never a fabricated PASS. **Leak-check:** 0 tolerated for *unbounded* growth — a bounded, plateauing warm-up is expected; continuous growth past the calibrated bound is a leak → FAIL.

Companion (§11.4.18) to the in-source documentation block at the top of the script. Both are updated in the same commit whenever the script changes.

Overview

§11.4.169 **memory** test-type coverage. It proves the LIVE Squid forward proxy (localhost:34128, container proxy-squid) does **not leak memory under sustained load**. It reads the proxy container's RSS at **baseline** (before any load), drives a **sustained soak** of $N \geq 200$ proxied requests over ≥ 30 s, samples RSS into a captured **time-series** along the way, re-reads RSS at the end, and PASSes only when the container **survived** the soak AND the final RSS did **not** grow past a documented, calibrated bound.

This is **distinct** from `tests/stress/proxy_forward_stress.sh` (which drives sequential + concurrent bursts and asserts request success + a latency distribution, but never measures memory) and from `tests/chaos/proxy_restart_recovery.sh` (fault-injection recovery). This suite is the only one that reads and asserts the proxy's **RSS**.

Prerequisites

- The dynamic proxy stack is up and the proxy-squid container is running (`podman ps` shows it). Absent → honest SKIP: `topology_unsupported`.

- podman (preferred, rootless per §11.4.161) OR docker on PATH for the READ-ONLY container introspection. Absent → honest SKIP.
- curl for the through-proxy load (READ-ONLY client use).
- Outbound reachability of the soak target through the proxy (default https://www.gstatic.com/generate_204). Unreachable via proxy **and** directly → honest SKIP:network_unreachable_external (external outage, not a proxy defect, §11.4.1).

Usage

Conductor invocation (host-safety caps per §12.6 / §11.4.169):

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 \
    bash tests/memory/proxy_memory_soak_test.sh
```

Longer, tighter soak (e.g. a release-gate calibrated run):

```
MEM_SOAK_REQUESTS=400 MEM_SOAK_MIN_SECONDS=60
    MEM_GROWTH_FACTOR=1.4 \
    GOMAXPROCS=2 nice -n 19 ionice -c 3 \
    bash tests/memory/proxy_memory_soak_test.sh
```

The script is authored to be **run by the conductor against the live stack** (§11.4.119 single-resource-owner: exactly one stream drives the proxy at a time). It never boots, stops, restarts, execs-into, or reconfigures any container.

How RSS is read (non-invasive — §11.4.128)

- **Primary:** `podman stats --no-stream --format '{{.MemUsage}}'` proxy-squid. `podman stats --no-stream` reads the container's **cgroup memory accounting counter ONCE and exits** — it does **not** exec into the container, send a signal, or spawn any process inside it, so it perturbs the squid processes not at all (the §11.4.128 observer-effect budget). It **aggregates every process in the container cgroup** (squid master + worker + helpers) — the correct leak signal. The human `used / limit` string's `used`-field is converted to a byte count (base-1000 kB/MB/GB and base-1024 KiB/MiB/GiB both handled).
- **Fallback (coarse):** if cgroup memory accounting is unavailable (some rootless cgroup-v1 hosts render `MemUsage` as `-`), the script reads the host-side `/proc/<pid>/status VmRSS` of the container's init PID via READ-ONLY `podman inspect --format '{{.State.Pid}}'`. This is the container **PID 1 only**, not the cgroup aggregate — documented as coarse; the primary path is preferred whenever available.

- **Genuinely unreadable** (both paths fail) → honest SKIP:topology_unsupported (memory accounting unavailable), **never** a fabricated 0 (§11.4.6).

Verdict logic

Condition	Verdict	Exit
proxy-squid absent / no container engine / RSS unreadable	SKIP:topology_unsupported	3
Soak served ≥ 1 request AND $\text{final_rss} \leq \text{baseline} \times \text{MEM_GROWTH_FACTOR}$	PASS (cites the RSS time-series)	0
Soak served ≥ 1 request AND $\text{final_rss} > \text{baseline} \times \text{MEM_GROWTH_FACTOR}$	FAIL — unbounded growth / leak	1
Container died during the soak	FAIL — possible OOM/crash	1
Soak served 0 requests BUT target reachable directly	FAIL — proxy defect (census void)	1
Soak served 0 requests AND target unreachable directly	SKIP:network_unreachable_external	3

Every PASS cites the captured `rss_timeseries.tsv` via `ab_pass_with_evidence` (§11.4.69) — never a metadata-only PASS.

Calibration (§11.4.6 — the bound is calibrated, not hardcoded-from-literature)

MEM_GROWTH_FACTOR (default 1.5) is the bounded-growth bound: the soak FAILs only when $\text{final_rss} > \text{baseline_rss} \times \text{MEM_GROWTH_FACTOR}$. 1.5 ($\leq 50\%$ growth over the soak) is a **conservative starting envelope**, deliberately loose so the first calibrated run does not false-FAIL on legitimate warm-up. The intended workflow:

1. Run once against a healthy proxy-squid. Read the captured `growth_ratio` in `soak.evidence` and the `rss_timeseries.tsv` shape.
2. A healthy proxy's time-series **rises early then plateaus** — the ratio settles to a stable value (commonly < 1.2). **Tighten** MEM_GROWTH_FACTOR to sit just above that observed steady-state

ratio (e.g. 1.3) for the release-gate run, so a real leak — which shows **continuous** growth across the samples — is caught.

3. `small_baseline` (`baseline < MEM_MIN_BASELINE_BYTES`, default 4 MiB) is surfaced in `soak.evidence`: below that floor the ratio is noise-prone (a tiny absolute delta reads as a large ratio). Squid's baseline is normally tens of MB, so the ratio test is the mechanical gate; the flag exists so an anomalous near-zero baseline is visible rather than silently trusted.

The point of the time-series (not just before/after) is exactly this: a leak is a **monotonic climb** across samples; a healthy proxy **plateaus**. Both are visible in `rss_timeseries.tsv`.

Outputs / evidence

Written under `qa-results/memory/proxy_soak_<ts>/` (gitignored; §11.4.128 raw corpus — only curated evidence is committed at release prep, §11.4.83):

- `rss_timeseries.tsv` — `sample_idx` `elapsed_s` `cumulative_requests` `rss_bytes` `rss_human`; the baseline is sample 0. **This is the PASS evidence.**
- `soak.evidence` — `baseline` / `final` / `delta` / `ratio` / `threshold` / `served-count` summary + the `OVERALL=` verdict line.
- `scratch/` — removed by the trap on every exit path (§11.4.14).

Environment knobs

Var	Default	Meaning
<code>HTTP_PROXY_URL</code>	<code>http://localhost:34128</code>	proxy the soak drives
<code>HTTP_PROXY_PORT</code>	34128	port for the listening probe
<code>MEM_SOAK_CONTAINER</code>	<code>proxy-squid</code>	container whose RSS is read
<code>MEM_SOAK_TARGET</code>	<code>https://www.gstatic.com/generate_204</code>	soak target
<code>MEM_SOAK_EXPECT</code>	204 200	accepted HTTP codes
<code>MEM_SOAK_REQUESTS</code>	240	request floor (the ≥ 200 soak floor)
<code>MEM_SOAK_MIN_SECONDS</code>	30	duration floor (the ≥ 30 s soak floor)

Var	Default	Meaning
MEM_SOAK_SAMPLES	8	RSS time-series points
MEM_SOAK_MAX_ROUNDS	60	runaway guard
MEM_GROWTH_FACTOR	1.5	bounded-growth bound — calibrate
MEM_MIN_BASELINE_BYTES	4194304	small-baseline surface floor (4 MiB)
CURL_MAX_TIME	20	per-request curl --max-time
MEM_SOAK_EVIDENCE_DIR	qa-results/memory/proxy_soak_<ts>	evidence dir

Edge cases

- **External outage** (proxy healthy, internet down) → SKIP, never a false-FAIL of a healthy proxy (§11.4.1).
- **Proxy dead / not serving** while the site is reachable directly → FAIL (real defect, memory census void; §11.4.68 no fail-open).
- **Container OOM/crash under load** → FAIL (the post-soak liveness re-check).
- **cgroup accounting unavailable** → honest SKIP:topology_unsupported.

Related scripts

- tests/lib/evidence.sh — sourced; provides ab_pass_with_evidence, ab_skip_with_reason, _code_in, port_is_listening.
- tests/stress/proxy_forward_stress.sh — §11.4.85 sustained + concurrent load with a latency distribution (no memory measurement).
- tests/chaos/proxy_restart_recovery.sh — §11.4.85 fault-injection recovery.

Last verified: 2026-07-01 (authored; parse-clean sh -n + bash -n; arithmetic self-checked; not executed here — the conductor runs it live against the proxy-squid container, §11.4.119).